

Laboratory 3: Process Scheduling

Anjali Kumari

September 2026

1 Objective

This laboratory implements and evaluates First In First Out (FIFO), Round Robin (RR), and a three-queue Multilevel Feedback Queue (MLFQ) scheduler. The experiments compare one processor with two processors, vary the Round Robin time quantum, and compare MLFQ with no periodic boost against a priority boost every 20 time units.

2 Workloads

The five professor-provided files are separate workload descriptions, not fragments to be concatenated. Each input line represents one process and follows

$$\text{arrival} \quad \text{CPU}_1 \quad \text{I/O}_1 \quad \text{CPU}_2 \quad \dots \quad -1.$$

Table 1: Professor-provided workloads

File	Processes	Workload characteristic
process1.txt	3	Large CPU-only bursts
process2.txt	4	Repeated 50 CPU / 5 I/O cycles
process3.txt	4	Short CPU bursts and frequent I/O
process4.txt	3	Mixed bursts with staggered arrivals
process5.txt	6	Larger mixed workload with staggered arrivals

3 Implementation and method

The simulator is compiled with C++17 and warning flags from the Makefile. It emits a per-CPU schedule, average turnaround time, maximum turnaround time, elapsed completion time including I/O, and total CPU busy time excluding I/O.

FIFO runs non-preemptively. RR is preemptive and was evaluated with quanta 1, 2, 4, 8, and 16. MLFQ has Q0, Q1, and Q2, with Q0 highest priority and a quantum of 2 in every queue. A process is demoted after using its quantum, and a newly ready higher-priority process preempts a lower-priority running process. MLFQ was evaluated with boost values 0 and 20. Every configuration was run with one and two CPUs, producing 80 experiment rows in `results.csv`.

4 How the scheduling algorithms work

4.1 FIFO

First In First Out, also called First Come First Served, keeps ready processes in arrival order. When a processor becomes free, it selects the process at the front of the ready queue. FIFO is non-preemptive in this simulator: once a process starts a CPU burst, it continues until that burst completes or the process blocks for I/O. This makes FIFO simple and predictable, but a long CPU burst can delay every process behind it.

4.2 Round Robin

Round Robin is a preemptive time-sharing algorithm. Ready processes are kept in a FIFO queue, but each process may run for at most a configured time quantum. If the CPU burst is not complete when the quantum expires, the process is preempted and placed at the back of the ready queue. A small quantum improves rotation and response for waiting processes but creates more interruptions; a large quantum reduces interruptions and approaches FIFO behavior.

4.3 Multilevel Feedback Queue

The MLFQ scheduler uses three priority queues, Q0, Q1, and Q2, with Q0 having the highest priority. Every queue uses a quantum of 2. New and newly awakened processes enter according to their current priority state. When a process consumes its quantum without completing its CPU burst, it is demoted to the next lower queue, up to Q2. A process in a lower queue can be preempted when a higher-priority process becomes ready. This is the key difference between MLFQ and ordinary Round Robin: the queue level reflects recent CPU behavior.

4.4 Periodic priority boost

Without a boost, a CPU-intensive process can remain in a lower queue for a long time. In the required boost experiment, every 20 time units all unfinished processes are returned to Q0 and their priority state is reset. The purpose is to limit starvation and give waiting lower-queue processes another opportunity. A boost can improve fairness without necessarily improving average turnaround, so both the numerical results and the schedule must be examined.

5 Results

5.1 How to read the results

The primary metric in this report is turnaround time. For process i ,

$$T_i = C_i - A_i,$$

where A_i is the arrival time and C_i is the completion time. The reported average is the arithmetic mean of all process turnaround times in one workload. Maximum turnaround is included because an average alone can hide starvation or a particularly late process.

The schedule plots should be read from left to right. A colored block means that the corresponding process occupied the CPU during that interval. A long block is evidence of a non-preemptive or large-quantum decision; many short blocks indicate frequent preemption or repeated returns from I/O. A gap means that every process was either blocked for I/O or had not arrived yet. Thus, the plots explain the numerical results rather than serving only as illustrations.

5.2 Algorithm comparison

Figure 1 compares average turnaround for one CPU using FIFO, RR with quantum 2, and MLFQ with a boost every 20 time units. FIFO is best for the CPU-heavy process1 workload because its long jobs are not repeatedly interrupted. The short and I/O-heavy process3 workload produces identical results for these settings because the processes frequently block for I/O.

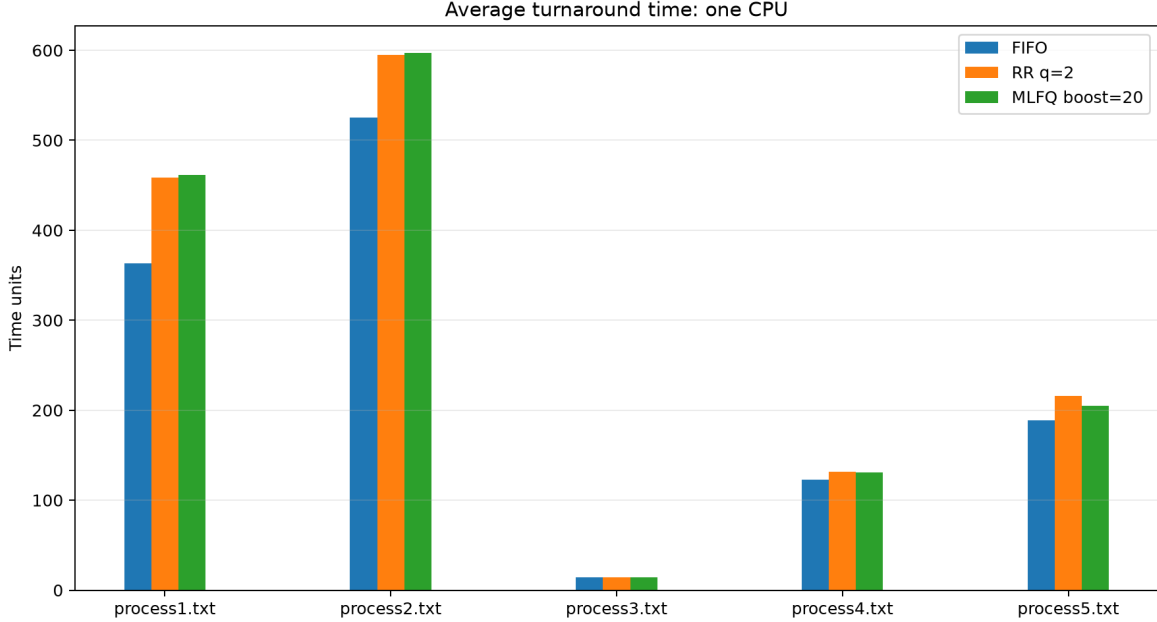


Figure 1: One-CPU algorithm comparison.

Table 2: One-CPU average turnaround for the main comparison

Workload	FIFO	RR ($q = 2$)	MLFQ (boost 20)
process1.txt	363.33	458.67	461.33
process2.txt	525.00	595.00	597.00
process3.txt	14.50	14.50	14.50
process4.txt	123.00	131.33	130.67
process5.txt	189.00	215.67	205.00

5.3 Workload-by-workload analysis

5.3.1 Process 1: long CPU-bound bursts

Process 1 contains three processes with long CPU-only bursts. Since there is no I/O, the ready queue is supplied only by the initial arrivals and the later process at time 10. FIFO completes this workload with an average turnaround of 363.33 on one CPU, compared with 458.67 for RR with $q = 2$ and 461.33 for MLFQ with boost 20. FIFO is better here because it avoids interrupting a long CPU burst that would eventually need the processor anyway. RR divides the long work into many slices, which gives earlier opportunities to other processes but delays the completion of the

first large jobs. MLFQ has similar behavior because CPU-bound processes are gradually moved to lower queues and continue to share the CPU.

The timeline figures make this trade-off visible. FIFO has large uninterrupted regions, while RR has a regular striped pattern. MLFQ initially favors new work in Q0, then separates processes according to their CPU use. The priority boost has no numerical effect for this workload in the tested configuration: both MLFQ variants have average turnaround 461.33. There are no I/O returns to create additional priority interactions, and the periodic reset does not change the final completion order enough to affect the measured average.

With two CPUs, FIFO improves from 363.33 to 230.00, a reduction of 133.33 time units. This is the clearest benefit of parallel execution in this workload because several long CPU bursts are available at the same time. RR and MLFQ also improve, but their one-CPU fairness mechanisms are less important once two processors can run independent jobs concurrently.

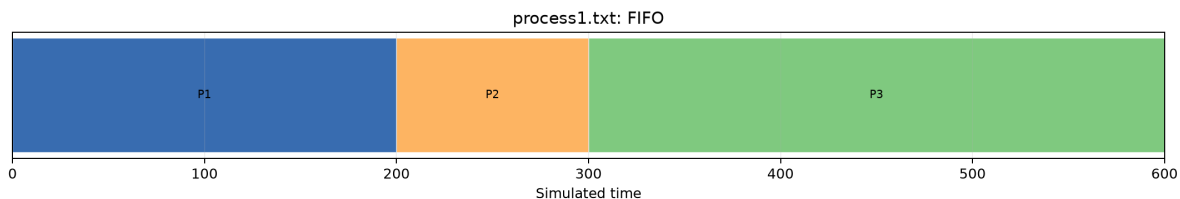


Figure 2: Process 1 under FIFO.

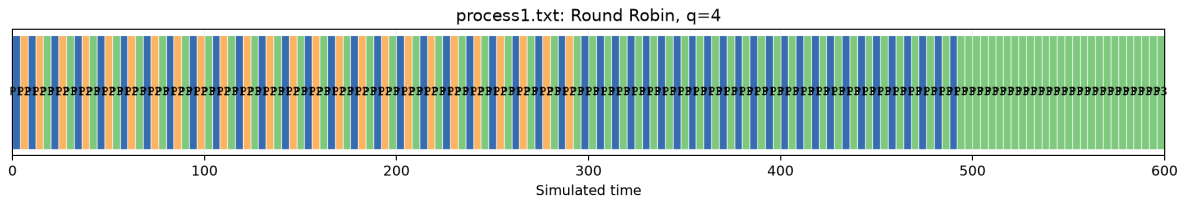


Figure 3: Process 1 under Round Robin with $q = 4$.

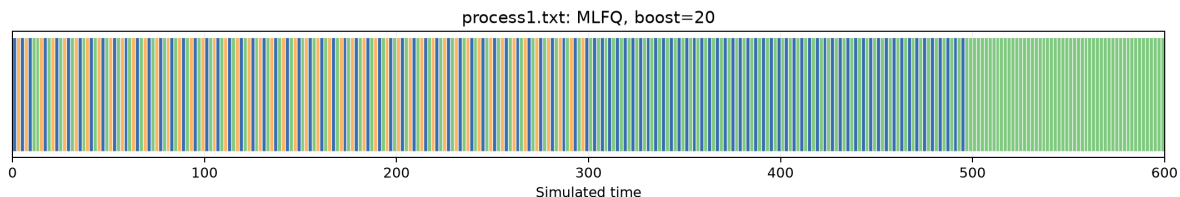


Figure 4: Process 1 under MLFQ with boost 20.

5.3.2 Process 2: repeated medium CPU and I/O bursts

Process 2 contains four similar processes, each alternating 50 units of CPU work with 5 units of I/O. FIFO has the best one-CPU average turnaround, 525.00, because the CPU bursts are long enough to keep the processor productive and the processes are structurally similar. RR with $q = 2$ gives 595.00, while MLFQ with boost 20 gives 597.00. The very small quantum causes many interruptions inside each 50-unit CPU burst, so fairness is gained at the cost of delaying completion.

The plot also shows why the difference is not simply a question of total CPU work. Every process eventually blocks for I/O, so the ready queue changes over time. A process that returns from I/O may re-enter a different position depending on the policy. Under MLFQ, the returning process retains scheduling state unless a boost resets it; the boost therefore changes queue order without reducing the total CPU demand. In this data, no boost is slightly better than boost 20: 595.00 versus 597.00. That result is not a contradiction of the purpose of boosting; it shows that fairness and average turnaround are different objectives.

Two processors reduce FIFO's average turnaround from 525.00 to 275.00. RR improves as well, but its best tested two-CPU value is 301.00 at $q = 2$, still above FIFO. The processors cannot execute I/O, so the improvement is bounded by the workload's blocking intervals. The timeline gaps and repeated returns explain why adding a second CPU helps substantially but does not halve every completion time.

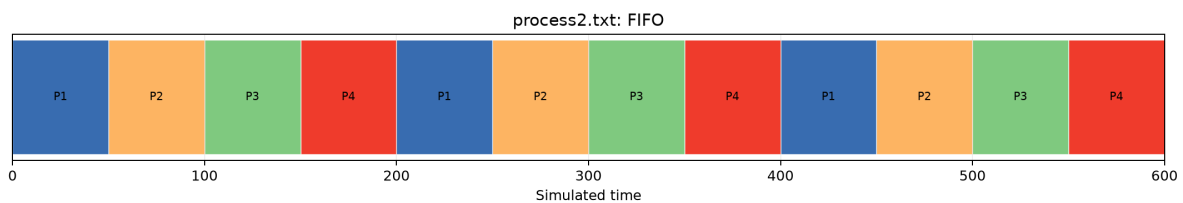


Figure 5: Process 2 under FIFO.

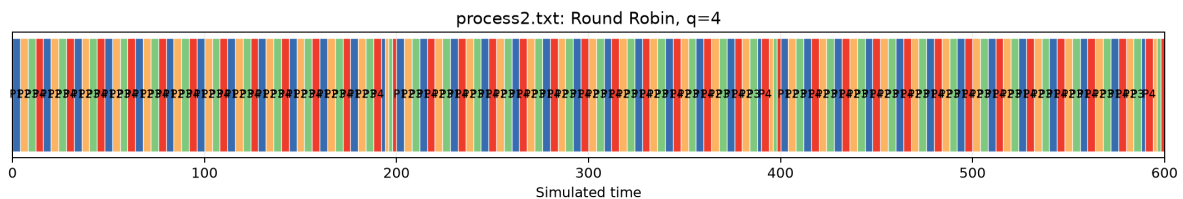


Figure 6: Process 2 under Round Robin with $q = 4$.

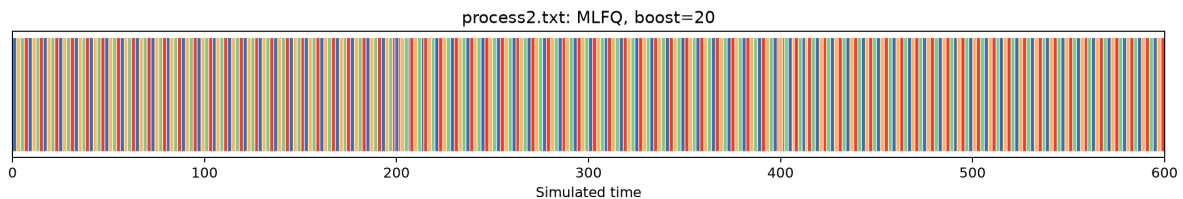


Figure 7: Process 2 under MLFQ with boost 20.

5.3.3 Process 3: very short CPU bursts

Process 3 is the strongest control case. Every CPU burst is one time unit and is separated by 5 units of I/O. Therefore, no RR quantum larger than one can split a CPU burst further, and MLFQ cannot meaningfully demote a process before its burst ends. All one-CPU configurations produce average turnaround 14.50 and maximum turnaround 16.00. The same invariance appears with two

CPUs, where the average is 13.50 for FIFO, every tested RR quantum, and both MLFQ boost settings.

This flat result is important evidence that the simulator is responding to workload structure. A quantum can only change behavior when a CPU burst lasts long enough to be preempted. Here, the process leaves the CPU after one tick, so the policy has almost no opportunity to reorder execution. The remaining turnaround is dominated by the 5-unit I/O delays and arrival/ready-queue timing rather than by scheduler choice.

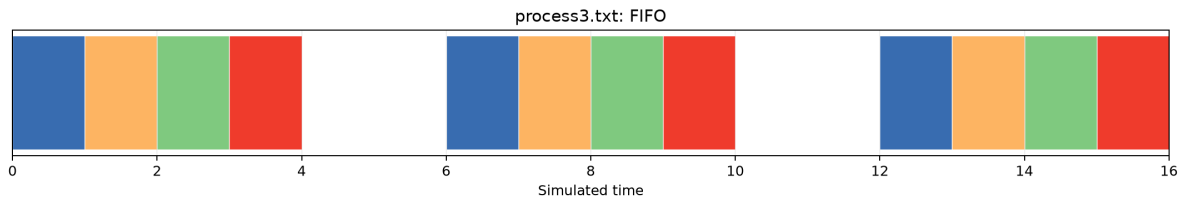


Figure 8: Process 3 under FIFO.

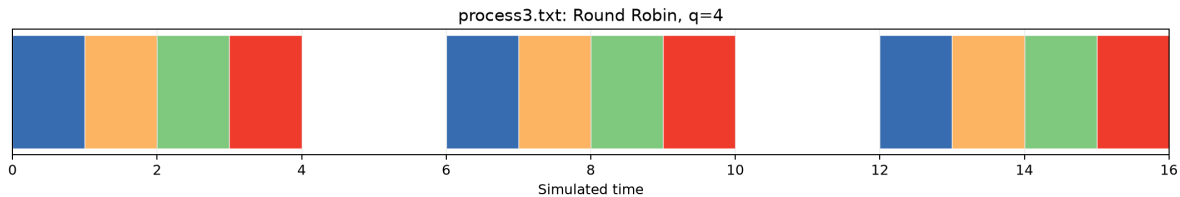


Figure 9: Process 3 under Round Robin with $q = 4$.

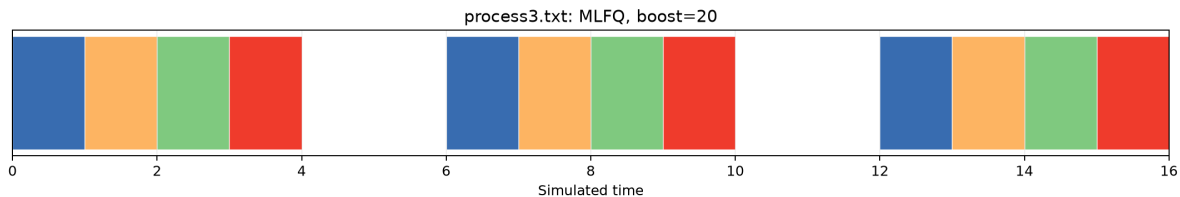


Figure 10: Process 3 under MLFQ with boost 20.

5.3.4 Process 4: staggered mixed workload

Process 4 contains long and short CPU bursts, repeated I/O, and arrivals at times 0, 10, and 15. This creates the first workload where the algorithms show distinct trade-offs. FIFO has the lowest one-CPU average, 123.00, because its non-preemptive execution completes long bursts efficiently. RR changes the result depending on quantum: the tested averages are 131.00, 131.33, 131.67, 128.33, and 129.00 for $q = 1, 2, 4, 8, 16$. The best value is 128.33 at $q = 8$, where the quantum is long enough to avoid excessive slicing but still permits useful interleaving.

MLFQ without boost gives 134.33, while boost 20 lowers it to 130.67. The improvement is modest but meaningful: periodically returning unfinished processes to Q0 prevents a process that has accumulated CPU history from remaining disadvantaged for too long. With two CPUs, MLFQ is strongest among the main configurations, at 78.33 without boost and 78.00 with boost 20. This

workload benefits from both parallel CPU service and priority responsiveness because processes arrive at different times and alternate between ready and blocked states.

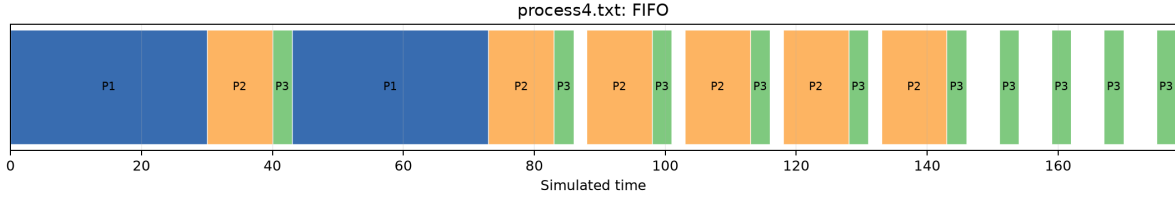


Figure 11: Process 4 under FIFO.

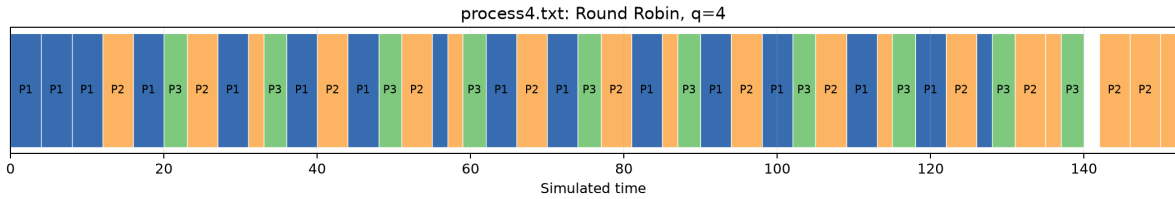


Figure 12: Process 4 under Round Robin with $q = 4$.

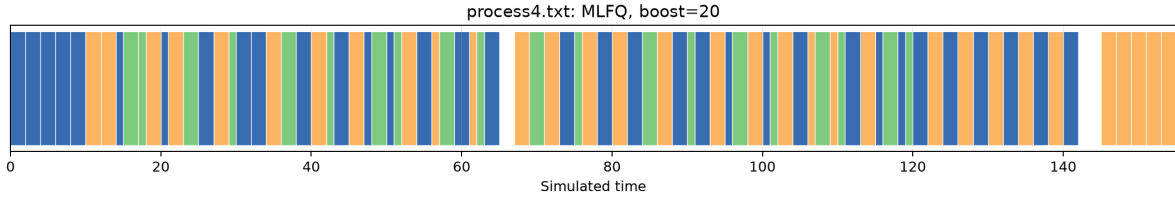


Figure 13: Process 4 under MLFQ with boost 20.

5.3.5 Process 5: mixed and highly bursty workload

Process 5 combines a long mixed process with several highly bursty processes arriving at time 15. It is the most demanding workload for a scheduler because long CPU bursts compete with short interactive-looking bursts. FIFO remains the best one-CPU baseline at 189.00, but it can delay newly arriving short work behind a long non-preemptive burst. RR responds more flexibly: its average is 205.50 at $q = 1$, 215.67 at $q = 2$, 207.17 at $q = 4$, 205.67 at $q = 8$, and 196.33 at $q = 16$. The relatively good result for $q = 16$ shows that a large quantum can be useful when the workload's burst boundaries already provide enough natural switching.

MLFQ improves from 210.17 without boost to 205.00 with boost 20. This is the largest one-CPU MLFQ improvement among the five workloads. The boost matters because the later bursty arrivals compete with processes that have already accumulated CPU time and moved down the queues. Resetting unfinished work to Q0 gives waiting processes more opportunities before the long-running work completes. With two CPUs, FIFO falls to 104.33 average turnaround, and the best RR result is 104.67 at $q = 16$. MLFQ is slightly higher at 108.50 with boost 20, showing that a policy designed for responsiveness does not necessarily minimize average completion time in every parallel workload.

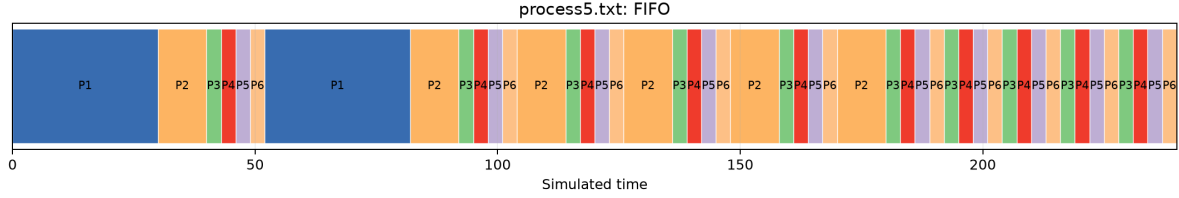


Figure 14: Process 5 under FIFO.

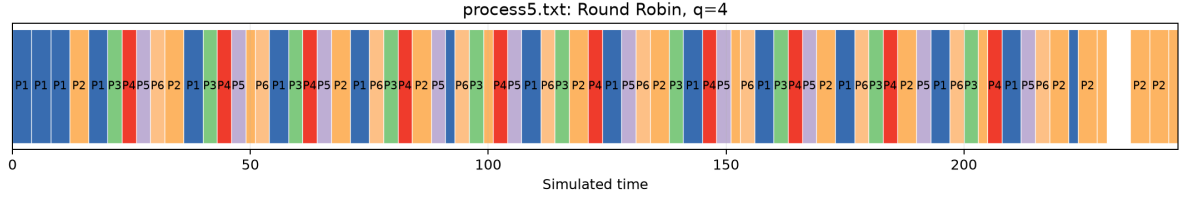


Figure 15: Process 5 under Round Robin with $q = 4$.

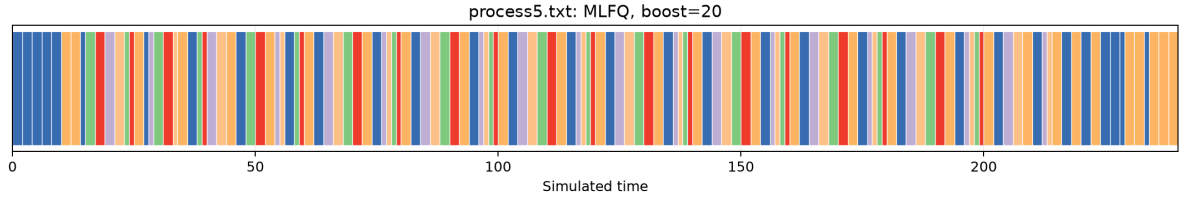


Figure 16: Process 5 under MLFQ with boost 20.

5.4 FIFO analysis

FIFO is the most useful baseline because it has no quantum and no priority state. A process remains on the CPU until its current CPU burst finishes or it blocks for I/O. This produces the clearest long contiguous blocks in the timeline figures. Its strength is low scheduling overhead in the simulated model and efficient completion of long CPU bursts. Its weakness is that a process arriving behind a long burst must wait, even if its own burst is short.

The results confirm that workload composition matters more than algorithm labels. FIFO is best on the CPU-heavy process1 and process2 workloads and on the mixed process4 and process5 workloads when average turnaround is the only objective. It is tied on process3 because one-unit CPU bursts eliminate most policy choices. FIFO should therefore be interpreted as an efficient throughput-oriented baseline, not as a universally fair policy. Maximum turnaround also matters: a policy can lower the average while still allowing one process to finish late.

5.5 Round Robin quantum analysis

The RR graph shows that there is no universally optimal quantum. At $q = 1$, every long CPU burst is split very aggressively, producing frequent queue rotations. Increasing the quantum reduces the number of rotations and lets a process make more progress per turn. However, a very large quantum approaches FIFO behavior and can delay newly ready processes.

The process3 curve is exactly flat because all CPU bursts are one unit. Process1 changes only slightly because it has no I/O-driven ready-queue competition. Process4 has its best value at $q = 8$, while process5 benefits most from $q = 16$ among the tested settings. These results demonstrate why the report must include both the graph and workload analysis: the same quantum can be a reasonable compromise for one workload and a poor choice for another.

5.6 MLFQ boost analysis

MLFQ combines feedback from CPU behavior with strict priority between Q0, Q1, and Q2. New processes begin in Q0. After using a quantum, a process is demoted, so CPU-intensive work gradually moves downward while short or newly returned work can receive quicker service. The priority boost experiment resets unfinished work to the highest queue at the configured interval.

The boost is beneficial when lower-queue work would otherwise wait behind repeatedly arriving or higher-priority work. This is visible in process4 and process5, where the one-CPU average improves by 3.66 and 5.17 time units respectively. It is neutral in process1 and process3, where the workload structure provides little opportunity for the boost to change the order. It slightly worsens process2, from 595.00 to 597.00, because resetting all unfinished work can disrupt an order that was already efficient for similar processes.

The correct conclusion is therefore not that boosting always improves turnaround. Its main purpose is to control starvation and long-term priority imbalance. Average turnaround is only one measurement, so a complete study should also inspect maximum turnaround and the timeline to see whether the boost improves fairness even when the mean changes little.

5.7 Two-processor analysis

Adding a second processor reduces waiting whenever at least two independent processes are ready. The largest reductions occur in CPU-heavy workloads: process1 FIFO falls from 363.33 to 230.00, and process2 FIFO falls from 525.00 to 275.00. Process3 changes only from 14.50 to 13.50 because its processes spend most of their time in I/O and its CPU bursts are already only one unit.

The two-CPU results also show why speedup is not exactly two. A processor cannot execute a process during I/O, and the workload may temporarily contain fewer ready processes than processors. In addition, policy decisions can leave one processor idle while a process is blocked or while queue order is being respected. The two-CPU timeline output should therefore be used together with elapsed runtime and CPU busy time: turnaround measures process experience, while busy time shows how much CPU work was actually available.

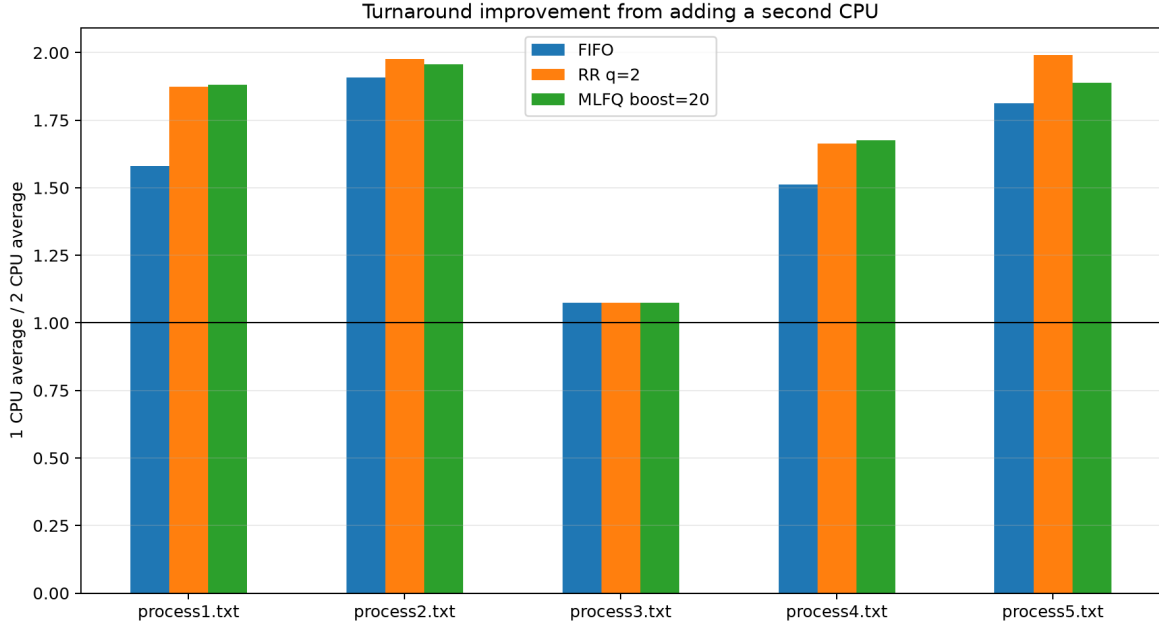


Figure 17: Effect of adding a second processor.

Figure 17 reports the ratio of one-CPU average turnaround to two-CPU average turnaround. Values above 1 indicate an improvement from adding a second processor. The improvement is strongest for workloads with enough runnable CPU work to keep both processors busy; workloads dominated by I/O or synchronization gaps benefit less.

5.8 Limitations and interpretation

These are discrete-event simulations with unit-time CPU execution and no modeled context-switch cost. Consequently, the graphs show scheduling decisions but do not measure real operating-system overhead. The study also uses five fixed workloads, so the conclusions describe these workload shapes rather than all possible programs. Finally, average turnaround should be read alongside maximum turnaround and the schedule: a lower average can coexist with poor fairness for one process.

5.9 Round Robin quantum

Figure 18 shows the RR quantum experiment. The tested quanta are 1, 2, 4, 8, and 16. The best quantum depends on the workload: process1 is nearly flat and favors a moderate-to-large quantum, process2 is best near $q = 2$, process3 is unchanged, and the mixed workloads respond more noticeably. This demonstrates the trade-off between responsiveness and context-switch frequency in the discrete simulator.

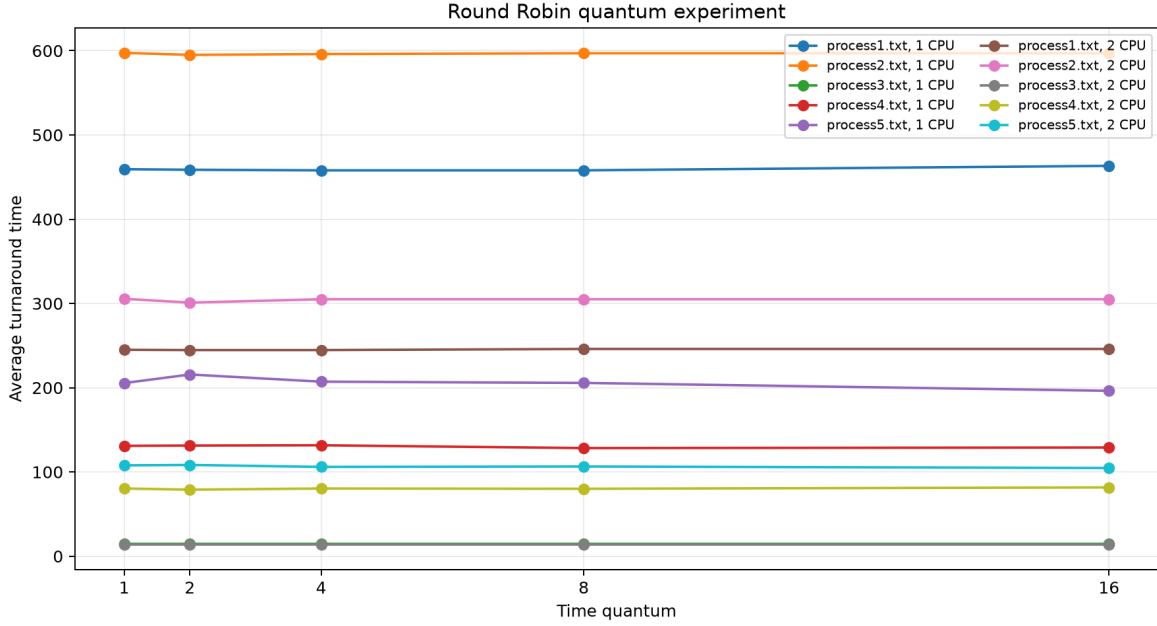


Figure 18: Round Robin average turnaround for quanta 1, 2, 4, 8, and 16.

5.10 MLFQ priority boost

Figure 19 compares MLFQ without a boost and with a boost every 20 time units. The boost improves average turnaround for process4 and process5 in the one-CPU experiments, by 3.66 and 5.17 time units respectively. It has no effect on process1 and process3 in the tested one-CPU cases, and it slightly increases turnaround for process2. A boost is therefore workload-dependent rather than universally faster.

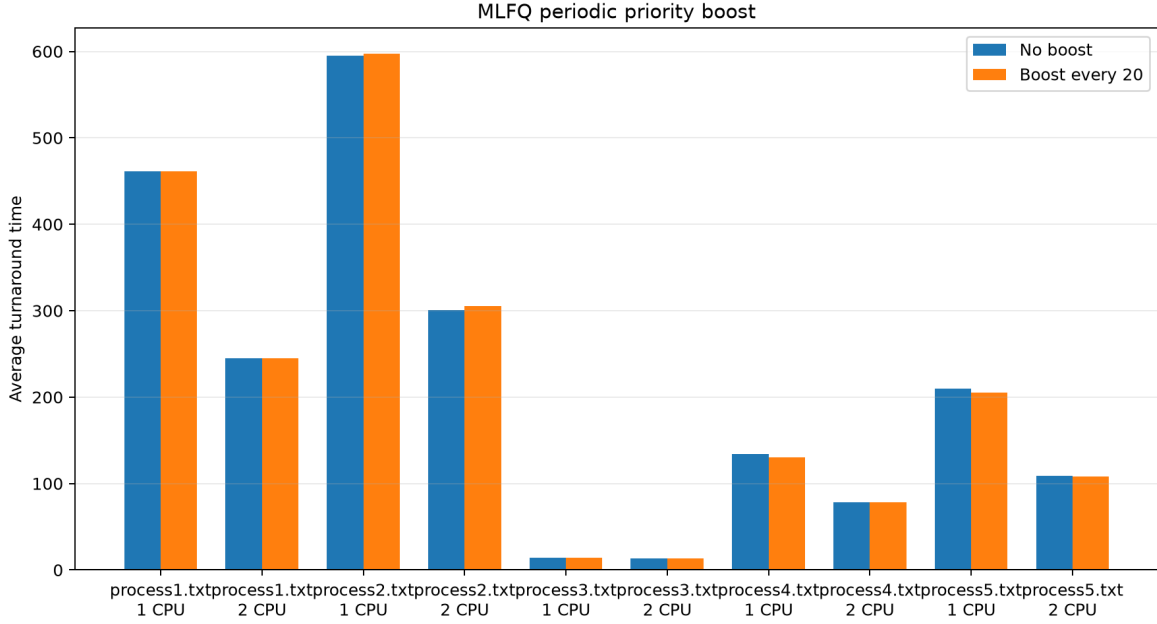


Figure 19: MLFQ average turnaround with and without periodic boost.

6 Conclusion

The workload shape strongly affects the best policy. FIFO performs well for long CPU-bound jobs in these experiments, while RR and MLFQ provide time sharing and respond differently to I/O and staggered arrivals. Periodic boosting can improve fairness and turnaround for mixed workloads, but it is not guaranteed to reduce the average for every workload. The second processor is most useful when sufficient independent CPU work is available.